Module 15 — Fine-Tuning & Alignment Operations

> **Goal:** Operate post-training as an engineering discipline: decide whether to train at all, run SFT with correct masking and run hygiene, scale training with parallelism and fault tolerance, apply preference optimisation and verifiable-reward RL without being fooled by proxies, distil safely, and gate every tuned model through evaluation and a registry. Complements Module 04 (PEFT mechanics) and Module 02 (data); this module is about operating runs, alignment methods and model lifecycle.

Table of Contents

- 15.1 Post-Training Decision and SFT Run Operations
- 15.2 Distributed Training Operations: Parallelism, Checkpointing, Fault Tolerance and Goodput
- 15.3 Preference Optimisation: RLHF, the DPO Family and Reward Models in Practice
- 15.4 RL with Verifiable Rewards (GRPO/RLVR): Environments, Rewards and Reward Hacking
- 15.5 Distillation Pipelines and Synthetic-Data Loops
- 15.6 Post-Training Evaluation, Regression Control and the Adapter/Model Registry

---

15.1 Post-Training Decision and SFT Run Operations

Conceptual Deep Dive

Training is the most expensive and least reversible lever, so climb a **decision ladder** and stop at the first rung that meets the target: prompt, then retrieval, then tools, then supervised fine-tuning (SFT), then preference optimisation, and only then continued pre-training. Fine-tuning changes behaviour and style well; it is a poor way to inject fast-changing facts such as company policies, which belong in retrieval (Module 12 section 1.4 decision trees). A readiness checklist: a defined evaluation, a measured baseline from the best prompted system, data of known quality and licence, and a serving plan (adapter or merged weights, 15.6).

SFT data mechanics decide whether training works. Render conversations with the exact chat template used at serve time (special tokens included); a mismatch is a classic silent failure (see the template-mismatch incident in the legacy modules). Apply a **loss mask** so only assistant tokens contribute to the cross-entropy: user and system text are context, not behaviour to imitate, and training on them teaches the model to generate user turns. In multi-turn data, mask user turns but keep tool-call and tool-response structure intact. **Sequence packing** concatenates short examples into one fixed-length row to avoid padding waste; it needs segment ids (or block-diagonal attention) and reset position ids so examples do not attend to each other, otherwise cross-contamination appears. Very short and very long examples mixed in a packed batch change the effective weight per example; normalise the loss deliberately (per token or per example) and know which you chose.

Hyperparameters (learning rate, epochs, LoRA rank and alpha, warmup, batch size) are starting points, not facts: full fine-tuning typically uses much smaller learning rates than LoRA, and one to three epochs often suffice for instruction data. Sweep small, on a subset, and let the evaluation choose.

Overfitting and forgetting show as a growing train/validation gap and a drop on general evaluations. Pick checkpoints by task and general evaluation, never by lowest loss (loss rewards fluency on the training distribution, not helpfulness). **Run hygiene:** a run manifest capturing config, seed, data hash, git SHA and

environment; resumable training; an experiment tracker. **Reproducibility** has limits (non-deterministic kernels, communication order), so define it as: same data and config give statistically equivalent evaluation results, within a measured seed-to-seed variance.

Cross-Connection Scenarios

- PEFT mechanics and rank selection (Module 04); data curation and decontamination (Module 02).
- Eval datasets and judge calibration used to pick checkpoints (14.2, 14.4).
- Registry record created at the end of the run (15.6).
- Serving the same chat template (Module 05).

Production Anti-Patterns

- Training on the eval set via leakage in synthetic data: scores jump, production does not.
- Picking the lowest-loss checkpoint: it is usually overfit and has lost general capability.
- Mismatched chat template at serve time: quality degrades with no error raised.

Minimal Working Code

A toy whitespace-tokeniser demonstration of chat-template rendering, an assistant-only loss mask (-100 labels) and sequence packing with segment ids.

```
IGNORE = -100
vocab = {}

def tid(tok):
    return vocab.setdefault(tok, len(vocab))

def render(messages):
    ids, labels = [], []
    for m in messages:
        header = [f"<|{m['role']}|>"]
        body = m["content"].split() + ["<|end|>"]
        train = m["role"] == "assistant"
        ids += [tid(t) for t in header + body]
        labels += [IGNORE] * len(header) + [tid(t) if train else IGNORE for t in body]
    return ids, labels

def pack(examples, max_len):
    packs, cur = [], {"ids": [], "labels": [], "seg": [], "pos":
[]}
    for n, (ids, labels) in enumerate(examples):
        if len(cur["ids"]) + len(ids) > max_len and cur["ids"]:
            packs.append(cur)
            cur = {"ids": [], "labels": [], "seg": [], "pos": []}
        cur["ids"] += ids
        cur["labels"] += labels
        cur["seg"] += [n] * len(ids)
        cur["pos"] += list(range(len(ids)))
    packs.append(cur)
    return packs

a = render([{"role": "user", "content": "hi there"}, {"role&
quot;: "assistant", "content": "hello friend"}])
b = render([{"role": "user", "content": "what is 2+2"}, {"ro
le": "assistant", "content": "4"}])
packs = pack([a, b], max_len=16)
p = packs[0]
print(p["labels"], p["seg"], p["pos"])
assert len(packs) == 1 and sorted(set(p["seg"])) == [0, 1]
assert all(l == IGNORE for l, i in zip(a[1], a[0]) if i in (tid("hi"), tid("there"),
tid("<|user|>")))
```

```
assert sum(l != IGNORE for l in p["labels"]) == 5       # 'hello friend <|end|>' and '4
<|end|>'
```

A warmup plus cosine learning-rate schedule, early stopping on an evaluation metric with patience, and a run-manifest writer capturing config, seed and data hash.

```
import hashlib, json, math, platform
import numpy as np

def lr_at(step, total, peak=2e-5, warmup=50, floor=0.1):
    if step < warmup:
        return peak * (step + 1) / warmup
    t = (step - warmup) / max(1, total - warmup)
    return peak * (floor + (1 - floor) * 0.5 * (1 + math.cos(math.pi * t)))

def early_stop(history, patience=3):
    """history: eval scores, higher is better. Returns (best_index, stop_now)."""
"
    best = int(np.argmax(history))
    return best, len(history) - 1 - best >= patience

def manifest(config, seed, data_bytes):
    return {"config": config, "seed": seed, "data_sha256": hashlib.sha256(d
ata_bytes).hexdigest(),
            "python": platform.python_version(), "numpy": np.__version__}

sched = [lr_at(s, 1000) for s in range(1000)]
assert abs(sched[49] - 2e-5) < 1e-12 and sched[0] < sched[49]
assert sched[-1] < sched[500] < sched[50]
assert early_stop([0.50, 0.58, 0.61, 0.60, 0.60, 0.59]) == (2, True)
m1 = manifest({"lr": 2e-5, "rank": 16}, 7, b"train-data-v1")
m2 = manifest({"lr": 2e-5, "rank": 16}, 7, b"train-data-v1")
assert m1 == m2
print(json.dumps(m1)[:120])
```

Interview Angle

Q: A team wants to fine-tune to teach the model company policies. What do you ask and recommend? **A:** I ask how often policies change, whether answers must cite sources, and what evaluation exists. Policies change and need citations, so I recommend retrieval with grounding first and SFT only for tone, format or tool behaviour. Trade-off: retrieval adds latency and index upkeep; tuning bakes stale facts into weights. Check: compare the best RAG prompt against a tuned model on a held-out policy question set, including after a policy edit.

Q: Why mask user tokens in the SFT loss? **A:** The goal is to learn assistant behaviour conditioned on the prompt, not to model user text. Unmasked training spends capacity on user distributions and can make the model emit user-style turns. Packing and multi-turn data make correct masks error-prone. Check: unit-test that labels outside assistant spans are -100, and compare evaluation with and without masking on a small run.

Q: Validation loss improves but product quality drops. Explain. **A:** Loss measures token likelihood on the training distribution, which can improve through memorised style while helpfulness, safety or tool-use degrade; templates may mismatch; the validation set may be near-duplicate of training. Trade-off: loss is cheap, task evaluation is costly. Check: choose checkpoints by task and general evaluations, inspect samples, and decontaminate the validation set.

15.2 Distributed Training Operations: Parallelism, Checkpointing, Fault Tolerance and Goodput

Conceptual Deep Dive

When a model or its optimiser state exceeds one GPU, training splits across devices, and each split buys memory at a communication cost. **Data parallelism** replicates the model and averages gradients. **FSDP and ZeRO** shard redundant state: stage 1 shards optimiser state, stage 2 also gradients, stage 3 (**ZeRO-3**, FSDP full sharding) also parameters, which are gathered

layer by layer on demand. **Tensor, pipeline and sequence parallelism** split the computation itself for models too large or contexts too long. Rule of thumb: prefer the simplest scheme that fits, because every added axis adds communication and debugging cost; tensor parallelism wants fast intra-node links.

Memory arithmetic. Mixed-precision Adam costs roughly 16 bytes per parameter before sharding (Module 01 section 2.5): 2 for bf16 weights, 2 for gradients, 12 for fp32 master weights plus two moments. ZeRO divides the sharded components by the data-parallel size. Activations come on top and scale with batch, sequence and depth; **activation checkpointing** recomputes them in the backward pass, trading about a third more compute for large memory savings. Global batch = micro-batch x gradient-accumulation steps x data-parallel size; change one and you must retune the learning rate or compensate.

Checkpointing. Write sharded, ideally asynchronous checkpoints so training does not stall. The cadence is a trade-off: frequent checkpoints cost overhead, rare ones lose work. The **Young/Daly** approximation gives the interval T ~ sqrt(2 x C x MTBF), where C is the checkpoint cost and MTBF is the mean time between failures of the whole job (which shrinks as GPU count grows).

Fault tolerance: health checks before launch, elastic restarts, straggler detection (one slow node gates the collective), and a deterministic, resumable dataloader that restores its position. **Goodput** is useful training time divided by wall-clock time; track it beside MFU (model FLOPs utilisation), loss, gradient norm and NaN or inf guards that trigger rollback to the last good checkpoint. **Cost control:** spot capacity suits fine-tuning with frequent checkpoints; right-size the GPUs rather than over-scaling.

Edge cases. A node dying mid-write corrupts a checkpoint: write to a temporary name, validate, then atomically rename, and keep the previous one. A loss spike right after resume usually means the dataloader or optimiser state did not restore (replayed or skipped data, reset moments).

Cross-Connection Scenarios

- PEFT memory savings versus full fine-tuning (Module 04).
- Spot-instance handling and capacity (11.6).
- Run manifest and resumability (15.1).
- Training-time failures as incident cases (Module 12).

Production Anti-Patterns

- Checkpointing only at epoch end: a failure late in a long epoch wastes hours of GPU time.
- No automated NaN or loss-spike rollback: bad updates are trained on for hours before a human notices.
- Scaling out GPUs without checking communication overhead: more devices, lower per-GPU efficiency, and a worse MTBF.

Minimal Working Code

A Young/Daly interval calculator and a Monte Carlo of goodput versus checkpoint interval under random failures.

```
import math
import numpy as np

def young_daly(ckpt_cost, mtbf):
    return math.sqrt(2 * ckpt_cost * mtbf)

def goodput(interval, work=72 * 3600, ckpt=60.0, mtbf=6 * 3600, restart=300.0, runs=20, seed=0):
    rng = np.random.default_rng(seed)
    out = []
    for _ in range(runs):
        elapsed, done = 0.0, 0.0
        next_fail = rng.exponential(mtbf)
        while done < work:
            seg = min(interval, work - done)
            need = seg + ckpt
            if elapsed + need <= next_fail:
                elapsed += need
                done += seg
            else:                               # failure: lose the segment, pay restart
                elapsed = next_fail + restart
                next_fail = elapsed + rng.exponential(mtbf)
        out.append(work / elapsed)
    return float(np.mean(out))
```

```
t_star = young_daly(60.0, 6 * 3600)
print("Young/Daly interval (s):", round(t_star))
res = {t: goodput(t) for t in (300, round(t_star), 7200, 28800)}
print({k: round(v, 3) for k, v in res.items()})
assert res[round(t_star)] > res[300] and res[round(t_star)] > res[28800]
```

A per-GPU memory estimator for DDP and ZeRO-1/2/3 with the 16 bytes per parameter breakdown (activations excluded; this is a stated limitation).

```
def per_gpu_gb(params_b, gpus, stage):
    p = params_b * 1e9
    weights, grads, optim = 2 * p, 2 * p, 12 * p          # bf16 weights, bf16 grads, fp32 master + 2 mom
ents
    if stage >= 1:
        optim /= gpus
    if stage >= 2:
        grads /= gpus
    if stage >= 3:
        weights /= gpus
    return (weights + grads + optim) / 1e9

print("params_B gpus  DDP   ZeRO-1  ZeRO-2  ZeRO-3  (GB/GPU, activations excluded)")
for params_b, gpus in ((7, 8), (70, 8), (70, 64)):
    row = [per_gpu_gb(params_b, gpus, s) for s in (0, 1, 2, 3)]
    print(f"{params_b:8d} {gpus:4d} " + " ".join(f"{v:7.1f}" for v in row))
assert abs(per_gpu_gb(7, 8, 0) - 112.0) < 1e-6
assert abs(per_gpu_gb(70, 8, 3) - 140.0) < 1e-6          # 70B full fine-tune does not fit 80 GB on 8
GPUs
assert per_gpu_gb(70, 64, 3) < 80
```

Interview Angle

Q: You can fine-tune a 70B model on 8 GPUs. Explain memory per GPU under ZeRO-3 versus QLoRA. **A:** Full fine-tuning state is about 16 bytes per parameter, roughly 1.1 TB, so even ZeRO-3 over 8 GPUs needs about 140 GB each before activations: too much for 80 GB cards without offload. QLoRA keeps 4-bit frozen weights (about 35 GB total, shardable) and trains small adapters, so it fits. Trade-off: QLoRA may lag full tuning on hard tasks. Check: measured peak memory and task evaluation of both.

Q: How often should you checkpoint a 3-day training run and why? **A:** Use Young/Daly as a starting point: T ~ sqrt(2 x checkpoint cost x MTBF). With a 60 s checkpoint and 6 h MTBF that is about 27 minutes; asynchronous checkpoints allow shorter intervals. Trade-off: overhead versus lost work. Check: simulate goodput across intervals using your measured failure rate and keep a rolling set of validated checkpoints.

Q: Training loss suddenly spikes at step 12,000. What is your playbook? **A:** Stop and inspect: gradient norm and NaN logs, the data batch at that step (corrupt or outlier data), learning rate and warmup, precision overflow, and whether a resume or hardware fault coincided. Roll back to the last good checkpoint, skip or fix the batch, lower the learning rate or tighten clipping. Check: the replay from the checkpoint reproduces or removes the spike, and loss tracks the pre-spike curve.

15.3 Preference Optimisation: RLHF, the DPO Family and Reward Models in Practice

Conceptual Deep Dive

After SFT, preference optimisation shapes behaviour that is easier to compare than to demonstrate: helpfulness, tone, refusal calibration. The classic pipeline collects preference pairs, trains a **reward model**, then optimises the policy against it with PPO-style RLHF (Ouyang et al., 2022). **DPO** (Rafailov et al., 2023) removes the explicit reward model and the RL loop by optimising the policy directly on preference pairs; variants such as IPO, KTO, ORPO and SimPO change the data format, remove the reference model or alter the objective to improve stability or data efficiency. Treat claims about which variant wins as task-dependent and verify on your own evaluation.

The DPO loss (Module 02 section 2.7) is -log sigmoid(beta x [(log pi(y_w) - log ref(y_w)) - (log pi(y_l) - log ref(y_l))]). The **reference model** anchors the policy, **beta** controls how far it may drift (small beta allows large movement, large beta keeps it near the

reference), and the implicit reward margin beta x (log-ratio difference) is a useful training diagnostic. DPO has a known **length bias**: longer responses accumulate larger log-probability differences, so models drift verbose; mitigate with length-normalised variants, length-controlled data and judging on length-matched pairs.

On-policy versus off-policy data. Pairs generated by a different model than the one being trained are off-policy; gains weaken as the policy moves away from the data distribution. Iterating (sample from the current policy, label, retrain) keeps data on-policy at higher cost.

Reward models are trained with the Bradley-Terry objective, -log sigmoid(r(chosen) - r(rejected)). Evaluate on held-out pairwise accuracy, calibration, and breakdowns by length and topic. Optimising a policy against a reward model is subject to **over-optimisation** (Goodhart): true quality peaks and then falls as proxy reward keeps rising, controlled with a KL penalty to the reference, early stopping and periodic fresh human or strong-judge evaluation.

Data quality dominates: judge bias, annotator disagreement (14.3) and verbosity bias; use ensembles, filter low-agreement pairs and report agreement. **Operational costs:** RLHF holds policy, reference, reward and often critic models in memory; DPO holds policy and reference. Add regression checks for sycophancy and safety, which preference tuning can worsen.

Edge cases. Pairs that differ only in length teach length, not quality; filter or balance them. Noisy labels call for label smoothing or conservative DPO-style variants.

Cross-Connection Scenarios

- Preference data generation and judge bias (Module 02, 14.2, 14.3).
- Reward-model-free alternatives with verifiable signals (15.4).
- Safety and sycophancy regression gates (15.6, 8.8).
- Memory footprint of reference models (15.2).

Production Anti-Patterns

- Running DPO before SFT has converged: the policy is far from the data distribution and the reference is poor.
- Using a judge from the same family as the policy and the data generator: self-preference bias inflates results.
- No held-out reward-model evaluation: you optimise against a proxy of unknown quality.

Minimal Working Code

The DPO loss and its gradient for a batch of (policy, reference) log-probability pairs, with a beta sweep showing implicit reward margins.

```
import numpy as np

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def dpo(pi_w, pi_l, ref_w, ref_l, beta):
    z = beta * ((pi_w - ref_w) - (pi_l - ref_l))            # implicit reward margin
    loss = -np.log(sigmoid(z)).mean()
    g = -beta * (1 - sigmoid(z)) / len(z)                   # d loss / d pi_w; d/d pi_l is -g
    return loss, g, z

rng = np.random.default_rng(0)
ref_w, ref_l = rng.normal(-20, 2, 8), rng.normal(-22, 2, 8)
pi_w, pi_l = ref_w + rng.normal(1.0, 0.5, 8), ref_l + rng.normal(-1.0, 0.5, 8)

for beta in (0.05, 0.1, 0.5):
    loss, g, z = dpo(pi_w, pi_l, ref_w, ref_l, beta)
    print(f"beta={beta}: loss={loss:.4f} mean_margin={z.mean():.3f}")

loss0, g, _ = dpo(pi_w, pi_l, ref_w, ref_l, 0.1)
eps = 1e-6
bumped = pi_w.copy(); bumped[0] += eps
num = (dpo(bumped, pi_l, ref_w, ref_l, 0.1)[0] - loss0) / eps
assert abs(num - g[0]) < 1e-5
assert dpo(pi_w, pi_l, ref_w, ref_l, 0.1)[0] < np.log(2)      # better than the untrained loss
```

A toy Bradley-Terry reward model fitted by logistic regression on feature vectors, reporting pairwise accuracy and the correlation between reward and length (a length-bias check).

```
import numpy as np

rng = np.random.default_rng(1)

def make_pairs(n):
    x_a = np.column_stack([rng.normal(size=n), rng.normal(size=n)])     # [quality, length]
    x_b = np.column_stack([rng.normal(size=n), rng.normal(size=n)])
    logit = 2.0 * (x_a[:, 0] - x_b[:, 0]) + 1.0 * (x_a[:, 1] - x_b[:, 1])   # annotators also like length
    a_wins = rng.random(n) < 1 / (1 + np.exp(-logit))
    chosen = np.where(a_wins[:, None], x_a, x_b)
    rejected = np.where(a_wins[:, None], x_b, x_a)
    return chosen, rejected

def fit(chosen, rejected, lr=0.5, steps=500):
    d = chosen - rejected
    w = np.zeros(d.shape[1])
    for _ in range(steps):
        p = 1 / (1 + np.exp(-(d @ w)))
        w += lr * (d * (1 - p)[:, None]).mean(axis=0)       # gradient ascent on log-likelihood
    return w

tr_c, tr_r = make_pairs(4000)
te_c, te_r = make_pairs(2000)
w = fit(tr_c, tr_r)
acc = float(np.mean((te_c - te_r) @ w > 0))
X = np.vstack([te_c, te_r])
corr = float(np.corrcoef(X @ w, X[:, 1])[0, 1])
print("weights", w.round(2), "held-out pairwise acc", round(acc, 3), "corr(rewar
d,length)", round(corr, 3))
assert acc > 0.7 and w[1] > 0.3 and corr > 0.2
```

Interview Angle

Q: Explain DPO versus PPO-based RLHF and when you would choose each. **A:** RLHF trains a reward model and optimises the policy with RL against it; DPO optimises a closed-form preference objective directly on pairs, which is simpler, cheaper and more stable. RLHF can exploit online sampling and a reusable reward signal; DPO needs good, ideally on-policy, pairs. I default to DPO for style and preference tuning and consider RLHF or RLVR when online exploration or a verifiable signal matters. Check: win rate against SFT on a held-out judge set, length-controlled.

Q: Your DPO model became verbose and sycophantic. Diagnose and fix. **A:** Likely length bias in the data or loss and preference labels rewarding agreeable answers. Check mean response length and win rate on length-matched pairs, and sycophancy probes. Fixes: length-normalised or length-controlled training, rebalanced pairs, higher beta, fewer steps, and adding disagreement examples. Trade-off: reduced win-rate headline numbers. Check: length-controlled win rate and a sycophancy suite before and after.

Q: How do you evaluate a reward model? **A:** Held-out pairwise accuracy against human labels, broken down by topic, length and difficulty; calibration of predicted preference probability; agreement with a strong judge; and a downstream test where policies optimised against it are evaluated by fresh humans. Trade-off: human labels are costly. Check: correlation of reward with human rating on policy samples at increasing KL, watching for the point where it turns down (over-optimisation).

15.4 RL with Verifiable Rewards (GRPO/RLVR): Environments, Rewards and Reward Hacking

Conceptual Deep Dive

RL with verifiable rewards (**RLVR**) replaces a learned reward model with a programmatic check, which removes a main source of over-optimisation noise. Where it applies: mathematics with checkable final answers, code with unit tests, outputs that must satisfy a schema or format, and constrained tool tasks where success is observable in environment state. Where it does not: open-ended writing or helpfulness, which still need preference methods (15.3).

GRPO (Shao et al., 2024, introduced in the DeepSeekMath work) simplifies PPO by dropping the learned value model. For each prompt, sample a group of completions, score them, and standardise rewards within the group: advantage = (r - group mean) / group standard deviation. Completions better than their siblings are pushed up, worse ones down. A KL penalty to a reference model and ratio clipping keep updates stable. A consequence: if all samples in a group succeed or all fail, advantages are zero and the prompt teaches nothing, so **curriculum and filtering** (drop prompts that are always solved or never solved, keep those with intermediate pass rates) raises sample efficiency.

Reward design. Outcome rewards score only the final result; process rewards score steps and need a process verifier. Partial credit (for example fraction of tests passed) gives denser signal than all-or-nothing; a small format reward teaches the output structure but overly strict formatting can stall learning because early samples rarely satisfy it. **Reward hacking** is the central failure mode: special-casing the tests, reading test files, exploiting parser quirks, printing the expected answer, or exploiting length. Counter with held-out tests the policy never sees, hidden variants, static checks and sandboxes, and audit high-reward samples by hand.

Infrastructure. Generation dominates cost, so use a fast inference engine for rollouts plus a trainer with weight synchronisation, and sandboxed execution for model-written code. Libraries (TRL, verl, OpenRLHF and others) change fast; check current docs.

Monitoring: mean reward, within-group reward variance, response length, entropy, KL to the reference, and a held-out evaluation that is separate from the reward environment. Stop on divergence: collapsing entropy, exploding length or KL, or held-out accuracy falling while reward rises.

Edge cases. Non-deterministic verifiers and flaky tests inject noise into rewards; make verifiers deterministic (fixed seeds, timeouts, retries only for infrastructure faults).

Cross-Connection Scenarios

- Reward models versus verifiable signals (15.3).
- Sandboxing and untrusted code execution (9.8, Module 08 section 3.9).
- Agent environments that double as RL environments (9.7).
- Distilling reasoning traces afterwards (15.5).

Production Anti-Patterns

- No held-out set separate from the reward environment: you cannot tell learning from exploitation.
- Executing model-written code outside a sandbox: an RL run becomes an arbitrary-code-execution service.
- Treating rising reward as proof of better behaviour: reward measures what the verifier sees, not what you want.

Minimal Working Code

Group-relative advantage computation and a toy categorical-policy gradient showing learning with GRPO-style advantages (no KL or clipping in this toy).

```
import numpy as np

def group_advantages(rewards, eps=1e-6):
    r = np.asarray(rewards, dtype=float)
    return (r - r.mean()) / (r.std() + eps)

assert np.allclose(group_advantages([1, 1, 1, 1]), 0)          # no signal when all succeed
adv = group_advantages([1, 0, 0, 0])
assert adv[0] > 0 > adv[1]

rng = np.random.default_rng(0)
logits = np.zeros(4)                                  # actions 0..3; action 2 is verifiably
correct
for step in range(300):
    p = np.exp(logits) / np.exp(logits).sum()
    actions = rng.choice(4, size=8, p=p)
    rewards = (actions == 2).astype(float)
    adv = group_advantages(rewards)
    grad = np.zeros(4)
    for a, A in zip(actions, adv):
        onehot = np.eye(4)[a]
        grad += A * (onehot - p)                      # d log p(a) / d logits
```

```
    logits += 0.3 * grad / len(actions)
p = np.exp(logits) / np.exp(logits).sum()
print(p.round(3))
assert p[2] > 0.9
```

A verifiable code reward: run candidate code against unit tests with a timeout and return a reward plus simple hack flags. This subprocess is a demonstration only; production needs a real sandbox (container or microVM, no network, resource limits).

```
import os, re, subprocess, sys, tempfile

TESTS = "assert solve(2) == 4\nassert solve(3) == 9\nassert solve(0) == 0\n"
EXPECTED = [4, 9, 0]

def code_reward(code, timeout=5):
    flags = []
    if re.search(r"sys\.exit|os\._exit|open\(|__import__|exec\(|eval\(", code):
        flags.append("suspicious_call")
    if all(re.search(rf"return\s+{e}\b", code) for e in EXPECTED):
        flags.append("hardcoded_outputs")
    with tempfile.TemporaryDirectory() as d:
        path = os.path.join(d, "cand.py")
        with open(path, "w") as f:
            f.write(code + "\n" + TESTS)
        try:
            ok = subprocess.run([sys.executable, "-I", path], cwd=d, capture_output=True,
                                timeout=timeout).returncode == 0
        except subprocess.TimeoutExpired:
            ok = False
            flags.append("timeout")
    return float(ok), flags

good = "def solve(x):\n    return x * x\n"
hard = "def solve(x):\n    if x == 2: return 4\n    if x == 3: return 9\n    return 0\n"
cheat = "import sys\nsys.exit(0)\n"
loop = "def solve(x):\n    while True:\n        pass\n"
print(code_reward(good), code_reward(hard), code_reward(cheat), code_reward(loop, timeout=1))
assert code_reward(good) == (1.0, [])
assert code_reward(hard)[1] == ["hardcoded_outputs"]
assert code_reward(cheat) == (1.0, ["suspicious_call"])
assert code_reward(loop, timeout=1) == (0.0, ["timeout"])
```

Interview Angle

Q: When is RL with verifiable rewards the right tool versus SFT or DPO? **A:** When success can be checked cheaply and reliably (maths, code, schemas, tool outcomes) and you want the model to explore strategies beyond demonstrations. SFT suits imitating known good outputs; DPO suits subjective preference. RLVR costs far more (rollout generation, sandboxes) and risks reward hacking. Check: held-out pass rate against an SFT baseline at equal compute, plus a hack audit of top-reward samples.

Q: Reward goes up, held-out accuracy goes down. What happened? **A:** Likely reward hacking or overfitting to the training prompts or verifier quirks: format gaming, test special-casing, length exploitation, or leakage of environment prompts into the held-out set. Inspect high-reward samples, compare hidden test variants, and watch entropy and length. Fixes: stronger verifiers, held-out tests, KL control, early stopping. Check: the gap between train reward and hidden-test accuracy after the fix.

Q: How do you sandbox reward computation for generated code? **A:** Run each candidate in an ephemeral container or microVM with no network, no secrets, read-only filesystem except a temp directory, CPU, memory and wall-clock limits, and process-count limits; return only structured results. Trade-off: startup latency and infrastructure cost, mitigated by pooled warm sandboxes. Check: an escape test suite (network, file reads, fork bombs) must fail inside the sandbox, with throughput measured.

15.5 Distillation Pipelines and Synthetic-Data Loops

Conceptual Deep Dive

Distillation transfers behaviour from a large teacher to a smaller, cheaper student. **Black-box** (sequence-level) distillation trains the student on teacher outputs and is the only option for API teachers; **white-box** distillation uses teacher logits or hidden states, giving a richer signal per token (a soft-label KL loss with temperature) but requiring access to the teacher and a shared tokenizer. **On-policy distillation** has the student generate and the teacher grade or correct those samples, which targets the student's own mistakes and reduces distribution mismatch.

Generation at scale needs prompt diversity (seed topics, templates, evolved prompts), sensible sampling temperature, **rejection sampling** with a programmatic verifier or a calibrated judge, near-duplicate removal, and decontamination against evaluation sets (Module 02). Quality filtering matters more than volume.

Legal and licensing. Teacher terms of use may restrict using outputs to train competing models (Module 02 scenario 4); read the current terms, and record data provenance (teacher MODEL_ID, prompts, licence, date) in the registry (15.6, 16.6).

Sizing and evaluation. Choose the student by latency and cost target, then measure quality retention against the teacher on your task: win rate versus the teacher, task metrics, and tail-slice performance. Account for cost honestly: teacher tokens for generation and verification, training compute and evaluation, versus per-request inference savings over the expected volume (break-even request count).

Model collapse. Training repeatedly on a model's own synthetic output narrows the distribution, loses tail behaviour and amplifies errors. Keep fresh real data in the mix, retain the original data across generations, and track diversity metrics (distinct-n, self-similarity, topic coverage). Distilling reasoning traces and tool-use trajectories is powerful but copies teacher mistakes and style; filter traces by correctness of the final answer. Any real data used as seed prompts carries privacy obligations (8.7).

Edge cases. Teacher mistakes copied with confidence: verify with programmatic checks rather than the teacher's own opinion. A student can match the teacher on easy items yet fail the tail, so stratify evaluation by difficulty.

Cross-Connection Scenarios

- Synthetic data generation and decontamination (Module 02).
- Student sizing, quantisation and serving cost (Module 04, 11.6).
- Verifiers shared with RLVR (15.4).
- Registry provenance and licence records (15.6).

Production Anti-Patterns

- Training on unfiltered teacher output: errors and refusals are copied at scale.
- Ignoring the teacher's licence: legal exposure appears after the model ships.
- Measuring success by loss only: low distillation loss does not imply task quality.

Minimal Working Code

A temperature-softened KL distillation loss between teacher and student logits with a temperature sweep.

```
import numpy as np

def softmax(x, t=1.0):
    z = np.asarray(x, dtype=float) / t
    z = z - z.max(axis=-1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

def distill_loss(teacher_logits, student_logits, t):
    p = softmax(teacher_logits, t)
    q = softmax(student_logits, t)
    kl = (p * (np.log(p + 1e-12) - np.log(q + 1e-12))).sum(axis=-1).mean()
    return (t ** 2) * kl                          # T^2 keeps gradient scale comparable across T

teacher = np.array([[4.0, 1.0, 0.5, 0.0], [0.2, 3.5, 0.1, 0.3]])
student = np.array([[2.0, 1.5, 0.5, 0.5], [0.5, 2.0, 0.5, 0.5]])
for t in (1, 2, 4):
    print(t, round(float(distill_loss(teacher, student, t)), 4))
assert distill_loss(teacher, teacher, 2) < 1e-9
assert all(distill_loss(teacher, student, t) > 0 for t in (1, 2, 4))
```

Diversity metrics (distinct-n, near-duplicate rate) for a generated set and a rejection-sampling filter using an injectable verifier.

```
def distinct_n(texts, n=2):
    grams, total = set(), 0
    for t in texts:
        toks = t.split()
        for i in range(len(toks) - n + 1):
            grams.add(tuple(toks[i:i + n]))
            total += 1
    return len(grams) / total if total else 0.0

def near_dup_rate(texts, thr=0.8):
    sets = [set(t.split()) for t in texts]
    dup = 0
    for i in range(len(sets)):
        for j in range(i):
            union = sets[i] | sets[j]
            if union and len(sets[i] & sets[j]) / len(union) >= thr:
                dup += 1
                break
    return dup / len(texts)

def rejection_filter(candidates, verifier):
    return [c for c in candidates if verifier(c)]

gen = ["the answer is 4", "the answer is 4", "two plus two equals four", &q
uot;answer: 4 because 2+2", "the answer is five"]
verifier = lambda s: "5" not in s and "five" not in s              # injected verifie
r stands in for a checker
kept = rejection_filter(gen, verifier)
print(len(kept), round(distinct_n(gen), 3), round(near_dup_rate(gen), 3))
assert len(kept) == 4 and near_dup_rate(gen) >= 0.2
assert distinct_n(["a b c", "d e f"]) == 1.0 and distinct_n(["a b c", "
;a b c"]) == 0.5
```

Interview Angle

Q: Design a pipeline to distil a large model into a 7B student for one task. **A:** Define the task evaluation and a teacher baseline; generate diverse prompts (from real, privacy-cleared traffic plus synthetic variations); sample teacher outputs; reject by programmatic verifier or calibrated judge; deduplicate and decontaminate; SFT the student, optionally with on-policy rounds; evaluate overall and on tail slices; record provenance and licence. Trade-off: teacher cost versus quality. Check: student reaches the target fraction of teacher quality on a held-out set at the latency goal.

Q: What can go wrong with training on synthetic data repeatedly? **A:** Model collapse: tails vanish, diversity shrinks, errors and stylistic quirks are amplified, and benchmark contamination can creep in. Mitigate with fresh real data in every mix, verifier-filtered samples, diversity metrics, and holding out evaluations. Trade-off: curation cost. Check: distinct-n and tail-slice accuracy per generation; stop when either degrades.

Q: How do you tell whether distillation was worth the cost? **A:** Compare total project cost (teacher tokens, training, evaluation, engineering) against inference savings per request times expected volume, net of any quality loss priced in. Compute the break-even request count and compare it with a cheaper alternative such as prompting a smaller model or routing. Check: measured cost per successful task and quality retention in a shadow or A/B run.

15.6 Post-Training Evaluation, Regression Control and the Adapter/Model Registry

Conceptual Deep Dive

A tuned model is promoted on evidence, not on the metric it was trained for. The **evaluation suite** covers: the target task; general capability retention (to detect **forgetting**); safety and refusal behaviour in both directions (harmful compliance and over-refusal); format and tool-use validity; long-context behaviour; and multilingual performance. A cheap model-diff check computes the logit KL between base and tuned models on a probe set (4.7) to see where behaviour moved. Use per-suite non-inferiority margins, not a

single blended score that lets a target gain hide a safety loss.

Forgetting mitigation: mix replay data from general distributions into the fine-tuning set, use a lower learning rate or lower LoRA rank, stop early, and test base-model abilities explicitly (for example instruction following, code, a held-out language).

The registry is the source of truth for model artefacts. Each record holds base-model hash, tokenizer hash, data hashes, training config and run manifest (15.1), evaluation report, owner, licence, intended use and known limitations. A state machine moves a model through draft, evaluated, staged, production and retired, and each transition requires specific evidence (evaluation report to leave draft; safety review and owner sign-off to stage; canary results and a rollback plan to reach production; a sunset notice to retire). Skipped states are rejected by code. At load time, check compatibility (base and tokenizer hashes match what the adapter was trained on), and verify signatures or provenance (13.1). Keep lineage so an auditor can walk from a production model back to data and code (16.3).

Multi-adapter serving operations (Module 05 section 1.7): route by tenant to an adapter version, hot-reload without restart, keep the previous version for rollback, and canary new adapters on a traffic slice. **Retirement:** define a sunset policy, notify dependent tenants, remove weights and caches, and track storage cost so dozens of stale adapters do not accumulate.

Edge cases. An adapter trained on base v1 and loaded on base v1.1 may degrade silently because shapes still match; compare base hashes, not names. A regression in a rare language not covered by the suite goes unseen; use traffic-derived slices and sample production outputs by language.

Cross-Connection Scenarios

- Judge calibration and evaluation datasets (14.2, 14.4).
- Red-team and safety regression corpus (8.8).
- Provenance, signing and lineage (13.1, 16.3).
- Canary rollout and rollback of served versions (Module 05, Module 10).

Production Anti-Patterns

- Promoting a tuned model on the target metric alone: safety and general capability regress unnoticed.
- Adapters with no lineage back to data and config: nobody can reproduce, audit or safely update them.
- No retirement process, so dozens of stale adapters accumulate with storage cost and security exposure.

Minimal Working Code

A registry state machine enforcing required evidence for each transition and rejecting skipped states.

```
ORDER = ["draft", "evaluated", "staged",
 "production", "retired"]
REQUIRED = {
    ("draft", "evaluated"): {"eval_report"},
    ("evaluated", "staged"): {"safety_report", "owner_signoff"},
    ("staged", "production"): {"canary_report", "rollback_plan"},
    ("production", "retired"): {"sunset_notice"},
}

class ModelRecord:
    def __init__(self, name, base_hash, tokenizer_hash, data_hashes):
        self.name, self.state = name, "draft"
        self.meta = {"base_hash": base_hash, "tokenizer_hash": tokenizer_hash, "
data_hashes": data_hashes}
        self.evidence, self.history = {}, ["draft"]

    def transition(self, to, evidence):
        key = (self.state, to)
        if key not in REQUIRED:
            raise ValueError(f"illegal transition {self.state} -> {to}")
        missing = REQUIRED[key] - set(evidence)
        if missing:
            raise ValueError(f"missing evidence: {sorted(missing)}")
        self.evidence.update(evidence)
        self.state = to
        self.history.append(to)
```

```
r = ModelRecord("support-adapter-v3", "sha-base", "sha-tok", ["sha-dat
a"])
r.transition("evaluated", {"eval_report": "r1"})
try:
    r.transition("production", {"canary_report": "c", "rollback_plan&q
uot;: "p"})
except ValueError as e:
    print("rejected:", e)
try:
    r.transition("staged", {"safety_report": "s"})
except ValueError as e:
    print("rejected:", e)
r.transition("staged", {"safety_report": "s", "owner_signoff": &q
uot;alice"})
assert r.state == "staged" and r.history == ["draft", "evaluated", "st
aged"]
```

A regression comparison across suites with per-suite non-inferiority margins and an aggregate forgetting score.

```
import numpy as np

def arr(rate, n):
    k = round(rate * n)
    return np.r_[np.ones(k), np.zeros(n - k)]

def noninferior(base, cand, margin, z=1.645):
    diff = cand.mean() - base.mean()
    se = np.sqrt(base.var() / len(base) + cand.var() / len(cand))
    return bool(diff - z * se > -margin), round(float(diff), 3)

SUITES = {   # name: (baseline, candidate, margin, is_general)
    "target_task": (arr(0.60, 2000), arr(0.72, 2000), 0.00, False),
    "general_qa":  (arr(0.80, 2000), arr(0.79, 2000), 0.05, True),
    "safety":      (arr(0.95, 2000), arr(0.88, 2000), 0.02, False),
    "rare_lang":   (arr(0.70, 300),  arr(0.69, 300),  0.05, True),
}
results, forgetting = {}, []
for name, (b, c, margin, general) in SUITES.items():
    results[name] = noninferior(b, c, margin)
    if general:
        forgetting.append(max(0.0, b.mean() - c.mean()))
print(results)
print("forgetting score", round(float(np.mean(forgetting)), 4))
promote = all(ok for ok, _ in results.values())
assert results["target_task"][0] and results["general_qa"][0]
assert not results["safety"][0] and not promote
```

Interview Angle

Q: How do you ensure a fine-tune did not damage safety behaviour? **A:** Run the red-team regression corpus and refusal suites (including over-refusal on benign prompts) against the base and tuned models with a non-inferiority margin, add safety examples to the training mix, and compare logit-KL on safety probes. Gate promotion in the registry on the safety report. Trade-off: extra evaluation time and tuning data. Check: attack success rate upper bound and benign false-refusal rate within the agreed margins.

Q: You serve 200 customer adapters. How do you manage lifecycle and rollbacks? **A:** Each adapter has a registry record with base hash and lineage, a per-tenant version pointer in a router, canary on a traffic slice, hot reload with the previous version kept warm, and automated retirement of unused versions with notice. Trade-off: storage and cache pressure versus fast rollback. Check: rollback time, adapter load latency, and per-adapter evaluation pass rate.

Q: What evidence must exist before promoting a tuned model to production? **A:** Evaluation report against baseline with margins for target, general, safety, format and multilingual suites; lineage (base, tokenizer, data, config hashes); licence and intended-use sign-off; canary results on real traffic; monitoring dashboards and a rollback plan; owner on call. Trade-off: slower release. Check: the registry rejects a transition when any item is missing.